PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

Today…

- Functions
  - Exercise: Collatz Conjecture
  - Pass By Reference vs. Pass By Copy
- Arrays and Vectors
- Discussion on HW4

Functions

Exercise: Collatz Conjecture

Pass by Reference vs Pass by Copy

Exercise – Collatz Conjecture

- Given a positive integer, perform the following operation each step:
  - If the number is even, divide it by two $x \rightarrow x/2$
  - If the number is odd, triple it and add one $x \rightarrow 3x + 1$
- The Collatz conjecture states that you will always reach 1.
- Write a program that computes how many steps it takes to reach 1 starting from a positive integer. The input and output should be:

```
Starting from X.
Step 1: X
Step 2: X
...
Step X: X
Starting from X, it took X steps.
```

Output example starting from 6

```
Starting from 6
Step 1: 3
Step 2: 10
Step 3: 5
Step 4: 16
Step 5: 8
Step 6: 4
Step 7: 2
Step 8: 1
Starting from 6, it took 8 Steps.
```

- Hints
  - Again, the `while` loop will be useful
  - This function is a ***procedure***, i.e., it will return `void`

UCLA

Passing Parameters by Reference

- The default behavior of passing parameters to a function is to "pass by copy"
  - Also called "pass by value"
- Parameters are copied to a local variable

```
bool isPalindrome(string str) {
        // ftn body ...
}

int main() {
    string huge_str = "ABC..xyz..123...\n\n...";
    if (isPalindrome(huge_str)) {
        // ...
    }
}
```

The string variable `huge_str` is copied to a local string variable `str`

- If the parameter has huge size (in terms of memory), then copying it is inefficient

UCLA

Passing Parameters by Reference

- Solution to this inefficiency → Use a reference!
- Recall: a reference is just another name of the variable, so unnecessary copies don't occur

```
bool isPalindrome(string& str) {
        // ftn body ...
}

int main() {
    string huge_str = "ABC..xyz..123...\n\n...";
    if (isPalindrome(huge_str)) {
        // ...
    }
}
```

str is now "*a reference to a string*" (type name) and it refers to huge_str

- However, as we've seen before, this is dangerous since str can be modified in the function, and we don't want this change to affect the original huge_str

UCLA

Passing Parameters by **Const** Reference

- Solution to this danger → Use a **const** reference!
- A const reference is a reference whose value cannot be modified (so it's safe)

```
bool isPalindrome(const string& str) {
        // ftn body ...
}

int main() {
    string huge_str = "ABC..xyz..123...\n\n...";
    if (isPalindrome(huge_str)) {
        // ...
    }
}
```

str is now "*a reference to a **const** string*" (type name) and it refers to huge_str

- Here, we pass a huge string object in a **safe** and **efficient** way
  - No unnecessary copies occurred
  - No changes in the original object

UCLA

Passing Parameters by Reference

- Passing by non-const reference can be used to change multiple parameters in a single function evaluation
- Say we have three variables a, b, c and want to increment them by two

```
int a = 2, b = 3, c = 4;
cout << a << ' ' << b << ' ' << c << endl;
add2(a, b, c);
cout << a << ' ' << b << ' ' << c << endl;
```

Desired output:

```
2 3 4
4 5 6
```

- What should be ***the signature of add2***?

```
void add2(int& x, int& y, int& z)
```

- Implementation?

```
{ x+=2; y+=2; z+=2; }
```

UCLA

Fill in the missing parts

- Fill in the missing part of the following code segment and predict the output

```
#include <iostream>
using namespace std;

double new_salary(double salary, double by)
{
        // FILL IN
}

void get_raise(double& salary, double by) {
         // FILL IN
}
```

```
int main() {
    double salary1 = 100, salary2 = 200;
    double raise_percent = 0.1;
    cout << "The old salaries are:\n"
        << salary1 << endl << salary2 << endl;

    // pass by value
    salary1 =
            new_salary(salary1, raise_percent);

    // pass by reference
    get_raise(salary2, raise_percent);

    cout << "The new salaries are:\n"
        << salary1 << endl << salary2 << endl;
    return 0;
}
```

UCLA

Vectors and Arrays

Vectors

- Vector is a special type of a class ("templated class") that can be considered an **array** of some other class type, with useful member functions
  - Recall that a "string" can be thought as an array of "char" variables, with useful member functions such as length, substr, etc.
- Must include <vector> library to use vectors
- Ex) Vector of `ints`
  - `vector<int> vint = { 2020, 5, 19 }; // contains 2020, 5, 19 in this order`
  - Access: subscript operator []
    - `cout << vint[0] << ", " << vint[1] << ", " << vint[2] << endl;`
- The vector class has many useful constructors
  - Creating a size "N" vector: `vector<int> vint2(N);`
  - Creating a size "N" vector and initialize all with "val": `vector<int> vint3(N, val);`
  - Creating a copy of other vector(copy constructor): `vector<int> vint4(vint);`

UCLA

Vectors

- What is the output of the code on the right?

- Ans: AAAAA
- Tvec is a vector of five “T” objects, where each of “T” objects is default-constructed

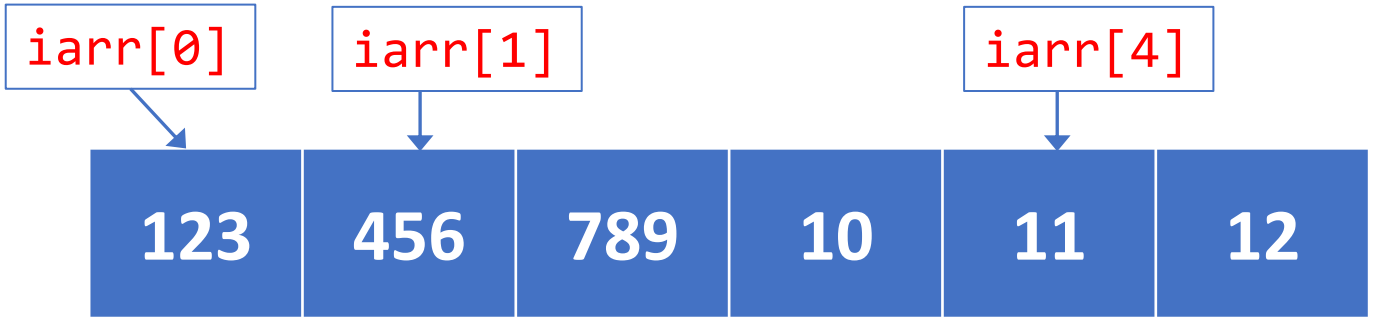
```
#include<iostream>
#include <vector>
using namespace std;
class T {
public:
    T() { cout << "A"; }
    T(int i) { cout << i; }
};
int main()
{
    vector<T> Tvec(5);

    return 0;
}
```

UCLA

Vectors and Arrays

- Vectors are like arrays, but with special features in addition
- In fact, (static) arrays are the most basic objects among “array-like” stuffs
- An int array of size 6 may look like (in the memory):



- “iarr” is the name of the array object
- This “iarr” actually “points to” the address of the part of the memory that stores the value 123 (or, iarr[0])
- Accessing the k-th element of iarr → iarr[k] (subscript operator)

Vectors and Arrays

- But (static) arrays cannot change in its size, and there are no member functions for array objects
  - It's why I called it the *most basic* "array-like" type
  - No push_back, insert, erase, etc.

- There is another type of arrays, "dynamic arrays" whose size can be changed dynamically
  - But not covered in this course, you will see them again in PIC 10B
  - Also this object is pretty difficult to deal with

- Vectors are much easier to handle, so in most cases you can just use vectors
  - Vectors = Arrays + Useful Features

UCLA

Discussion on HW4

HW4

- Exercise 1:
  - Just do the computation :)
- Exercise 2:
  - I shared the solution to the Num2Rome solution before
  - You can get some hints from “Strategy A”
  - How does this change with the function calls?
  - Hint: Use the idea of using the helper variable “romans” in the previous exercise

UCLA

Your Feedback is welcome

- Don’t hesitate to give a feedback on the discussion
- You can use a link in BruinLearn (Google Form)

Click this link

At this point, you may want to request
“more hw hints”,
“more challenging examples/exercises”,
“more details on the classroom examples”, etc.
Please let me know what you want!